



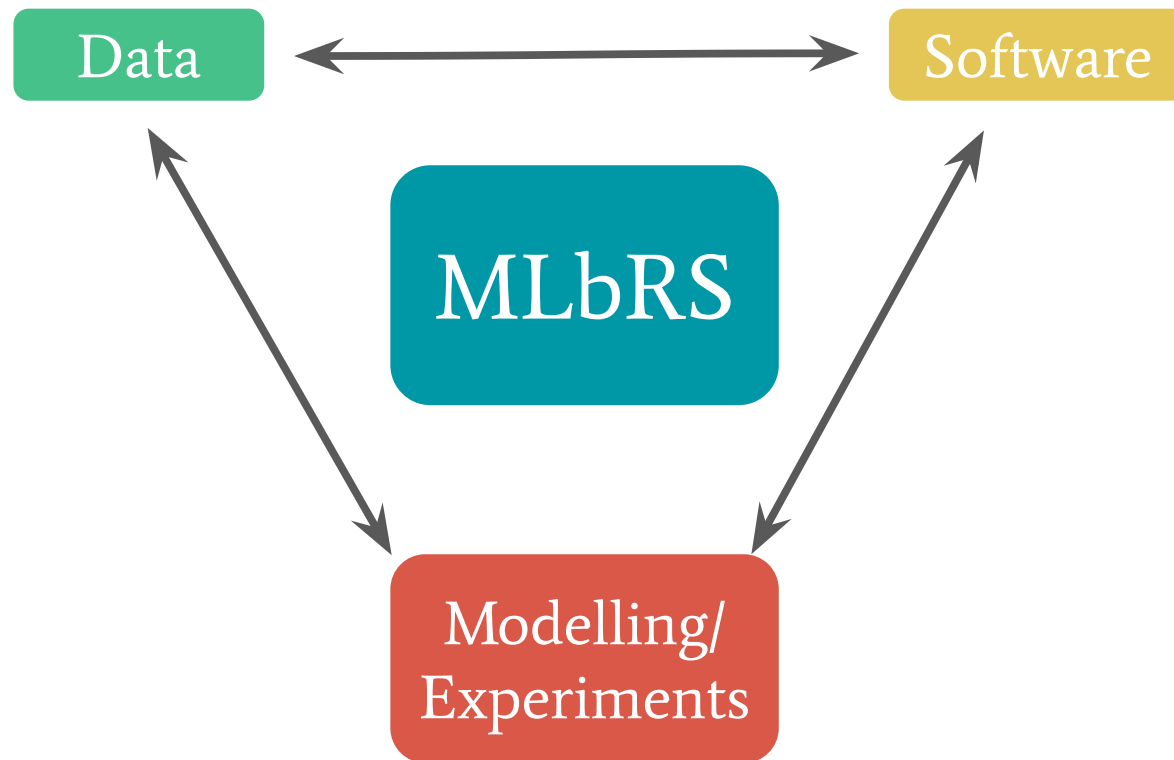
SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

5. Reproducibility in MLbRS in practice

Machine-learning based research software



Established principles for sustainable software

- **S**ingle responsibility principle: One task per code entity
- **O**pen-closed principle: Open to extension, closed to change
- **L**iskov substitution: Allow substitution with derived version
- **I**nterface segregation: Only depend on things you need
- **D**ependency inversion: Depend on abstractions, not concretes

Apply to all aspects of MLbRS!

Separation of concerns/ Single responsibility

In software engineering

- Responsibility = 'reason to change'
- Any code entity should just have one reason to change
- Any entity implements only things concerned with a single problem

In MLbRS

- Every part has one 'reason to change'
- data, code and parameterization handled by different entities
- Raw, preprocessed, final data are distinct and don't mix
- every step in the pipeline does one thing only
- library code and experiments in separate repositories

Mental model : "functional purity" - no side effects, same input => same output

Open/closed principle

In software engineering

- new features can be added without modifying existing code
- existing code is stable even if new functionality is added
- established features have baseline correctness => tests

In MLbRS

- make use of modularity in your models, pipeline
- add new experiment runs without impacting previous ones
- Bind data + model def. to experiment

Practical tool : Use frameworks, version control (data + code)

Liskov substitution principle

In software engineering

- system should be able to run with derived classes without knowing it
- 'Design by contract'
- introduces guarantees: everything that adheres to a certain behavioral spec works

In MLbRS

- each stage in the pipeline corresponds to an abstract contract
- different versions of a stage => different implementations of its contract
- use a different model, different data transforms etc without touching our setup
- Guarantees allow faster iteration

Tool: Polymorphism, config templates, repo templates

Interface segregation

In software engineering

- don't force entities to depend on unused interfaces
- split large interfaces into smaller modular parts
- protects from undesired side effects
- sparsifies dependency graph

In MLbRS

- no unnecessary interdependencies between pipeline steps
- make pipeline minimal and specific: only pass data, model modules, evaluators needed
- can help performance too!

Practical tip : use workflow/experiment managers

Dependency inversion

In software engineering

- Functionality should depend on abstractions, not implementations
- introduces loose coupling
- relationships are one-way: abstraction -> implementations

in MLbRS

- build pipeline on interacting abstractions
- then use those to derive implementations
- allows for easy scaling and testing

Tool: Encapsulation and abstraction

Summary

- SOLID principles from software engineering
- useful for all aspects of MLbRS
- introduce guarantees and invariants => reduce degrees of freedom in system

Compromises are sometimes necessary:

- Performance: CUDA likes doing a lot at once -> violate separation of concerns
- Not all projects need everything: scale, goal, persistence
- Problem space evolves with experiments, often no static 'target' => not everything everywhere all at once
- 'By the spirit, not by the letter'

Practical example I

- have a library git repository + git experiment repository
- library: code, tests, pip releases, CI/CD (later)
- structure of experiment repo:
 - src: code
 - configs: parameterization
 - one config per run/sub-experiment
 - data: 'hot data'. This can be elsewhere too (directory, drive, network, cloud storage)
 - results: produced artifacts: config files, model snapshots, evaluation metrics

Practical tip : Use templates/cookiecutter for repository setup

Library: <https://github.com/ssciwr/cookiecutter-python-package>

Experiments: <https://github.com/drivendataorg/cookiecutter-data-science>

Practical example I

Done:

- Ensured reproducibility by saving parameterization together with results
- Code + dependencies under version control
- Parameterization and code separated
- stable library code separate from changing experiment code

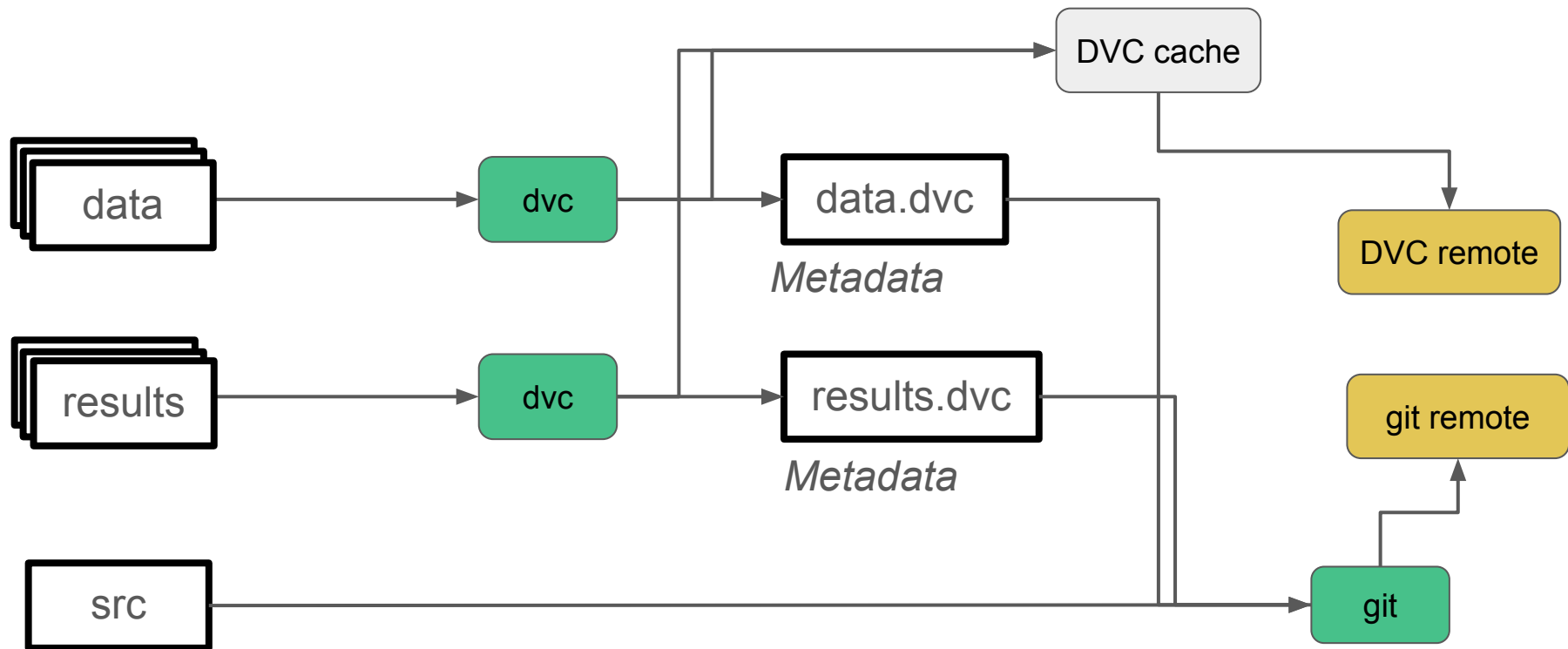
Not done:

- Separation of concerns:
 - training logic, output setup, config handling all in one script
- Interface segregation:
 - Everything depends on full config
 - No easy substitution of data processing without changing main script...
- Manual versioning of experiments
- guarantees about behavior **only in software part!**

Practical example II

- MLbRS = Data + Code + Model => Version control for experiments?
- Automatic versioning of processed data, experiments, pipeline stages?
- using 'git for data' isn't ideal because
 - Git cannot diff binary files
 - gitlfs is often limited (github)
 - many big files make repositories big
 - separation of concerns: data should not be the concern of the source code VC
- possible solution: DVC (data version control)
 - <https://dvc.org>
 - adds a git-like metadata layer to get 'git for data and artifacts'
 - tracks the metadata with git
 - Handles data in content-addressable storage

DVC mental model



Practical example II:

Setup dvc repository and add data

- Initialize dvc repository inside experiment git repository:

```
dvc init
```

```
=> .git, .dvc
```

- Add data to data/raw directory

```
dvc get https://huggingface.co/datasets/ylecun/mnist ./mnist -o  
./data/raw/ylecun/mnist
```

```
=> raw data, not tracked by dvc
```

```
dvc add ./data/raw/ylecun/mnist
```

```
dvc commit
```

Practical example II: Define pipeline

- Stage 1 - Data preprocessing

- Apply a set of transforms to raw data
- Save transformed data to disk
- Input: dataset name, dataset subset size
- Output: transformed data



- Stage 2 - model training

- Read preprocessed training data
- Train model



- Stage 3 - model evaluation

- Load trained model
- Run on test data
- Record metrics

```
dvc stage add -n prepare_data \
```

```
-p data.name,data.train_size,data.test_size \
```

```
-d src/prepare_data.py -d data/raw/${data.name} \
```

```
-o data/processed/${data.name} \
```

```
python3 ./src/prepare_data.py --dataset ${data.name} --train_size
```

```
${data.train_size} --test_size ${data.test_size}
```

```
stages:
  preparation:
    cmd: python3 ./src/prepare_data.py --dataset ${data.name} --train_size ${data.train_size} --test_size ${data.test_size}
    deps:
      - ./src/prepare_data.py
      - ./data/raw/${data.name}
    params:
      - data.name
      - data.train_size
      - data.test_size
    outs:
      - data/processed
```

Practical example II: Run pipeline

- Visualize defined pipeline with various backends

`dvc dag`



- Reproduce current pipeline

`dvc repro`

- Use defined parameters to run the pipeline again
- Relies on data being available!
- Will skip stages for which the input/parameters has not changed

Practical example II: Manage experiments

Run pipeline with a modified set of parameters

```
dvc exp run --set-param configs/training.yaml:learning_rate=0.003  
--set-param data.train_size=12000
```

- Modify parameter in params.yaml or referenced sub-configs
- Caches all current experiments
- Allows us to select some and commit it for further use, or branch on them
- DVC will keep track of experiments automatically: dvc.lock
- **Commit dvc.lock asap!**

Practical example II: Track and visualize results

- A pipeline stage can output 'metrics' to quantify e
- **dvc exp show**

Experiment	Created	mean_loss	min_loss	max_loss
workspace	-	0.056889	0.0027314	0.16338
main	12:49 PM	0.083361	0.011967	0.19013
└─ 20142e5 [duddy-yews]	01:02 PM	0.056889	0.0027314	0.16338
└─ e34fee6 [bendy-clip]	01:02 PM	0.085162	0.0022099	0.20693
└─ f46371e [amuck-fish]	12:57 PM	0.073541	0.0014057	0.2323
└─ 6df4e2f [mangy-plot]	12:57 PM	0.083361	0.011967	0.19013

```

training:
  cmd: python3 ./s
  deps:
    - ./src/train_
    - data/process
  params:
    - configs/mode
    - data.name #
    - configs/trai
  outs:
    - results/config_used.yaml
    - results/trained_model.pth
  metrics:
    - results/training_log.csv:
      cache: false
      You, 2 minutes ago • Uncommi

results
  ! config_used.yaml
  {} results.json
  ≡ trained_model.pth
  ≡ training_log.csv

evaluation:
  cmd: python3 ./src/test_model.py --model ${model_co
  deps:
    - ./src/test_model.py
    - results/trained_model.pth
  params:
    - configs/model.yaml: # colon at the end makes thi
    - data.name # this is a simple parameter from para
    - configs/test.yaml: # colon at the end makes this
  metrics:
    - results/results.json:
      cache: false
  
```

Practical example II: Manage experiments

- Select experiment to apply and continue with

```
dvc exp apply *experiment_name*
```

- Apply parameters to the config file
- **You must create new git commit with the changes!**

- Promote experiment to git branch to iterate further

```
dvc exp branch *experiment_name*
```

- What you want to keep must go into version control!
- Commit changes often!

Practical example II: DVC Remotes

- Like git, dvc supports remotes to store tracked artifacts
- Remotes can be
 - Http address with login
 - SSH server
 - S3-based cloud storage
 - ...
 - local

```
dvc remote add -d default_local ../path/to/local/remote
```

```
dvc commit, dvc push, dvc pull, dvc update...
```

- Add data to cache
- Push the data cache to remote
- Update data cache from remote
- Share repository **and** data remote with coworkers to share experiment status

Practical example II - summary

DVC approach is more SOLID

- Pipeline definition separate from code (Separation of concerns)
- Pipeline definition separate from pipeline parameterization
- Stage defined by command + parameters, not code (Liskov subst., Dependency inversion)
- Arbitrarily composable (Interface segregation)
- Extension is easy, even if current stages' code is unknown (Open/closed principle)
- Experiment management doesn't have to be hardcoded or manually managed
- Much more to it:
 - DVCLive Python module, e.g., for usage with jupyter notebooks
 - DVC extension for vscode
 - Support for [hydra.cc](https://doc.dvc.org/user-guide/experiment-management/hydra-composition): <https://doc.dvc.org/user-guide/experiment-management/hydra-composition>